

Deep Q-Network

José Pablo Kiesling Lange - 21581

```
In [1]: import os
import matplotlib.pyplot as plt
import numpy as np
from collections import deque
import random

import gymnasium as gym
from gymnasium.wrappers import RecordVideo

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.losses import MeanSquaredError

In [2]: os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
tf.get_logger().setLevel("ERROR")

In [3]: gpus = tf.config.experimental.list_physical_devices('GPU')
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
    except RuntimeError as e:
        print(e)

In [4]: SEED = 327
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

In [5]: env = gym.make("CartPole-v1", render_mode="rgb_array")
```

El entorno de CartPole-v1 tiene un espacio de acciones discreto:

Acción	Descripción
0	Mover el carro a la izquierda
1	Mover el carro a la derecha

Y el espacio de estados es continuo, con los siguientes valores:

Estado	Descripción	Mínimo	Máximo
Cart Position	Posición del carro	-4.8	4.8
Cart Velocity	Velocidad del carro	-Inf	Inf
Pole Angle	Ángulo del poste	-24°	24°
Pole Velocity	Velocidad angular del poste	-Inf	Inf

Q-Network

Dado que la entrada de la red será el estado, tendrá 4 nodos de entrada. La salida será la acción a tomar, por lo que tendrá 2 nodos de salida.

```
In [6]: state_size = env.observation_space.shape[0]
action_size = env.action_space.n

In [7]: class DQNAgent:
    def __init__(self, state_size=state_size, action_size=action_size):
        self.state_size = state_size
        self.action_size = action_size

        self.replay_buffer = deque(maxlen=10000)

        self.learning_rate = 0.001
        self.gamma = 0.99
        self.epsilon = 1.0

        self.epsilon_min = 0.01
        self.epsilon_decay = 0.995

        self.train_steps = 0
```

```

self.target_update_every = 1000

self.model = self.build_model()
self.target_model = self.build_model("Target")

self.target_model.set_weights(self.model.get_weights())

def build_model(self, type="Main"):
    model = Sequential(name="QNetwork_{}".format(type))

    model.add(Dense(128, input_dim=self.state_size, activation='relu'))
    model.add(Dense(128, activation='relu'))
    model.add(Dense(self.action_size, activation='linear'))

    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss=MeanSquaredError()
    )

    model.summary()

    return model

def remember(self, state, action, reward, next_state, done):
    self.replay_buffer.append((
        state.squeeze(), action, reward, next_state.squeeze(), float(done)
    ))

def act(self, state):
    if np.random.rand() <= self.epsilon:
        return np.random.randint(self.action_size)

    q_values = self.model.predict(state.squeeze()[None, :], verbose=0)

    return int(np.argmax(q_values[0]))

def replay(self, batch_size):
    minibatch = random.sample(self.replay_buffer, batch_size)
    states, actions, rewards, next_states, dones = zip(*minibatch)

    states = np.vstack(states).astype(np.float32)
    next_states = np.vstack(next_states).astype(np.float32)
    actions = np.array(actions, dtype=np.int32)
    rewards = np.array(rewards, dtype=np.float32)
    dones = np.array(dones, dtype=np.float32)

    q_online = self.model.predict(states, verbose=0)

    q_next_target = self.target_model.predict(next_states, verbose=0)
    max_next = np.max(q_next_target, axis=1).astype(np.float32)
    targets = rewards + self.gamma * (1.0 - dones) * max_next

    q_target_full = q_online.copy()
    q_target_full[np.arange(batch_size), actions] = targets

    history = self.model.fit(states, q_target_full, epochs=1, verbose=0)

    if self.epsilon > self.epsilon_min:
        self.epsilon *= self.epsilon_decay

    self.train_steps += 1
    if self.train_steps % self.target_update_every == 0:
        self.target_model.set_weights(self.model.get_weights())

    return {"loss": float(history.history["loss"][0])}

```

In [8]: agent = DQNAgent()

Model: "QNetwork_Main"		
Layer (type)	Output Shape	Param #
=====		
dense (Dense)	(None, 128)	640
dense_1 (Dense)	(None, 128)	16512
dense_2 (Dense)	(None, 2)	258
=====		
Total params: 17,410		
Trainable params: 17,410		
Non-trainable params: 0		
Model: "QNetwork_Target"		
Layer (type)	Output Shape	Param #
=====		
dense_3 (Dense)	(None, 128)	640
dense_4 (Dense)	(None, 128)	16512
dense_5 (Dense)	(None, 2)	258
=====		
Total params: 17,410		
Trainable params: 17,410		
Non-trainable params: 0		

Train

```
In [9]: batch_size = 32
num_episodes = 320

In [10]: returns_history = []
window = 20

In [11]: env.reset(seed=SEED)
env.action_space.seed(SEED)

Out[11]: 327

In [12]: for episode in range(num_episodes):
    obs, info = env.reset()
    state = obs.astype(np.float32)

    episode_return = 0.0

    for t in range(500):
        action = agent.act(state)

        next_obs, reward, terminated, truncated, info = env.step(action)
        next_state = next_obs.astype(np.float32)
        done = terminated or truncated

        agent.remember(state, action, reward, next_state, done)

        if len(agent.replay_buffer) > batch_size:
            agent.replay(batch_size)

        state = next_state

        if done:
            break

        episode_return += reward

    returns_history.append(episode_return)

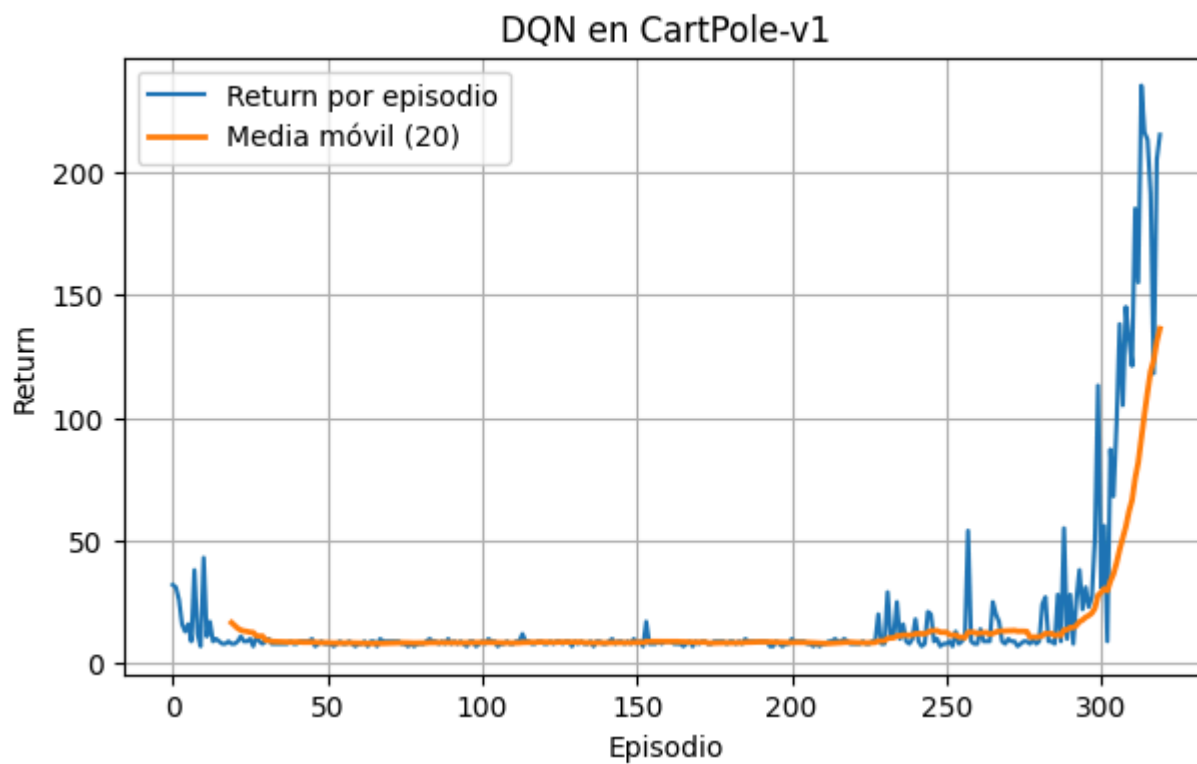
    if (episode + 1) % 50 == 0:
        avg = np.mean(returns_history[-10:])
        print(f"Ep {episode + 1:4d} | Ret: {episode_return:6.1f} | Avg(10): {avg:6.1f} | ε: {agent.epsilon:6.3f}")

Ep  50 | Ret:   8.0 | Avg(10):   8.6 | ε:  0.047
Ep 100 | Ret:   8.0 | Avg(10):   8.2 | ε:  0.010
Ep 150 | Ret:  10.0 | Avg(10):   8.8 | ε:  0.010
Ep 200 | Ret:   8.0 | Avg(10):   8.1 | ε:  0.010
Ep 250 | Ret:   8.0 | Avg(10):  11.7 | ε:  0.010
Ep 300 | Ret: 113.0 | Avg(10):  36.7 | ε:  0.010
```

Evaluation

```
In [13]: plt.figure(figsize=(7,4))
plt.plot(returns_history, label="Return por episodio")
if len(returns_history) >= window:
    ma = np.convolve(returns_history, np.ones(window)/window, mode='valid')
    plt.plot(range(window-1, window-1+len(ma)), ma, linewidth=2, label=f"Media móvil ({window})")
plt.xlabel("Episodio"); plt.ylabel("Return")
plt.title("DQN en CartPole-v1")
plt.grid(True); plt.legend(); plt.show()

print(f"Return promedio últimos 20: {np.mean(returns_history[-20:]).1f}")
print(f"Epsilon final: {agent.epsilon:.3f}")
```



Return promedio últimos 20: 136.2
Epsilon final: 0.010

```
In [14]: def evaluate_greedy(env, agent, n_episodes=10):
total = 0.0
for _ in range(n_episodes):
    obs, _ = env.reset()
    state = obs.astype(np.float32)
    ep_ret = 0.0
    while True:
        q = agent.model.predict(state[None, :], verbose=0)
        action = int(np.argmax(q[0]))
        obs, reward, terminated, truncated, _ = env.step(action)
        state = obs.astype(np.float32)
        ep_ret += reward
        if terminated or truncated:
            break
    total += ep_ret
return total / n_episodes
```

```
In [15]: avg_eval = evaluate_greedy(env, agent, n_episodes=10)
print(f"Evaluación promedio (greedy, 10 eps): {avg_eval:.1f}")
```

Evaluación promedio (greedy, 10 eps): 299.0